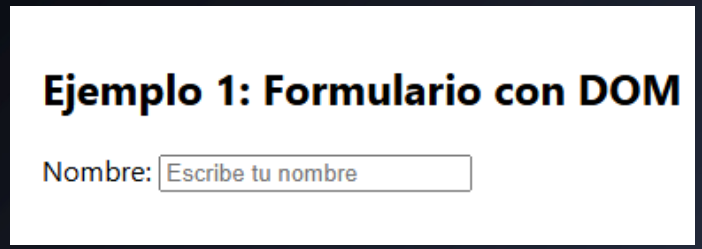
3. DOM Declarativo y REACT

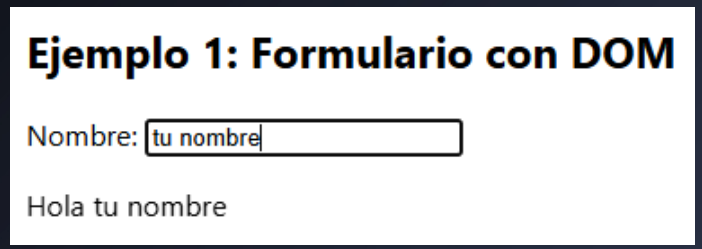
Eduardo Delgado
DDINT – 2 DAM

Los datos en DOM imperativo Vs Declarativo

En el DOM imperativo, los datos NO están en el HTML, si no en javascript.

Ejemplo 1:






```
<script>
const input = document.getElementById("nombre");
const mensaje = document.getElementById("mensaje");

// Nuestro "estado"
let estado = { nombre: "" };

input.addEventListener("input", function () {
  estado.nombre = input.value;  // 1) Cambia el dato
  actualizarInterfaz();          // 2) Actualiza el DOM
});

function actualizarInterfaz() {
  mensaje.textContent = "Hola " + estado.nombre;
}
</script>
```

En este caso declaramos *el objeto "estado"* que guarda los datos insertados por el usuario en el *campo nombre* para luego mostrarlo como *mensaje*

El objeto estado tiene una propiedad nombre cuyo valor está vacío. Un método input.addEventListener(...) se encarga de detectar el evento "input" , cambia el valor y lanza una función actualizarInterfaz() {...} que actualiza el DOM mostrando el mensaje.

Los datos en DOM imperativo Vs Declarativo

En el DOM imperativo, los datos NO están en el HTML, si no en javascript.

Ejemplo 1:

Ejemplo 1: Formulario con DOM

Nombre: tu nombre

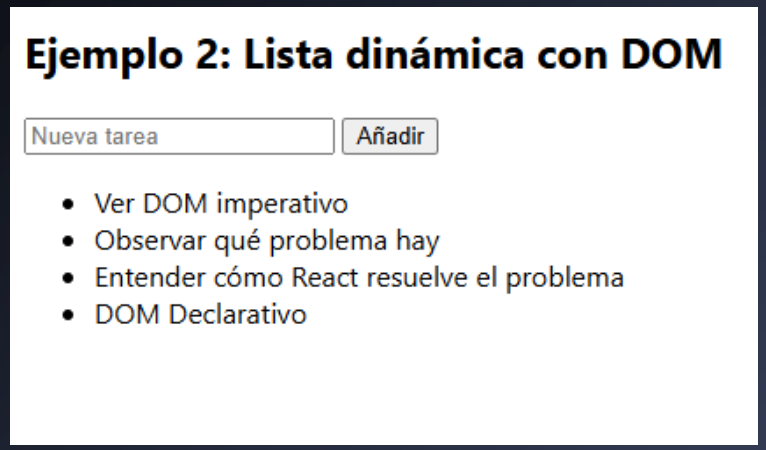
Hola tu nombre

```
<script>
const input = document.getElementById("nombre");
const mensaje = document.getElementById("mensaje");

// Nuestro "estado"
let estado = { nombre: "" };

input.addEventListener("input", function () {
  estado.nombre = input.value;  // 1) Cambia el dato
  actualizarInterfaz();          // 2) Actualiza el DOM
});

function actualizarInterfaz() {
  mensaje.textContent = "Hola " + estado.nombre;
}
</script>
```

- **El usuario escribe en el campo.**
- **JavaScript guarda lo que escribe en estado.nombre.**
- **Luego JavaScript cambia el texto del <p id="mensaje">.**
- **Nosotros controlamos manualmente ese cambio en el DOM**

Los datos en DOM imperativo Vs Declarativo

En el DOM imperativo, los datos NO están en el HTML, si no en javascript.
Ejemplo 2:




```
<script>
const input = document.getElementById("tarea");
const btn = document.getElementById("btnAdd");
const lista = document.getElementById("lista");

// Nuestro estado
let tareas = [];

btn.addEventListener("click", function () {
  tareas.push(input.value);  // 1) Cambia el dato
  input.value = "";
  renderLista();             // 2) Actualiza el DOM
});

function renderLista() {
  lista.innerHTML = "";  // Borramos todo

  for (let i = 0; i < tareas.length; i++) {
    const li = document.createElement("li");
    li.textContent = tareas[i];
    lista.appendChild(li);
  }
}
</script>
```

En este caso declaramos *el array "tareas"* que guarda los datos insertados por el usuario en el *campo de texto* para luego mostrarlo como *una lista*

El array tareas está inicialmente vacío. El método btn.addEventListener(...) se encarga de detectar el evento "click" , y mediante una función añade al array el **valor** insertado. Después reinicia el campo de texto y por último lanza una función renderLista() {...} que actualiza el DOM mostrando la lista de tareas.

Los datos en DOM imperativo Vs Declarativo

En el DOM imperativo, los datos NO están en el HTML, si no en javascript.

Cada vez que cambian los datos, *tenemos que volver a dibujar la lista en pantalla*

lista.innerHTML = “” ; borra TODAS las tareas visibles en pantalla, aunque solo hayamos añadido una nueva. Estamos destruyendo y recreando la interfaz cada vez

```
<script>
const input = document.getElementById("tarea");
const btn = document.getElementById("btnAdd");
const lista = document.getElementById("lista");

// Nuestro estado
let tareas = [];

btn.addEventListener("click", function () {
  tareas.push(input.value);  // 1) Cambia el dato
  input.value = "";
  renderLista();             // 2) Actualiza el DOM
});

function renderLista() {
  lista.innerHTML = "";  // Borramos todo

  for (let i = 0; i < tareas.length; i++) {
    const li = document.createElement("li");
    li.textContent = tareas[i];
    lista.appendChild(li);
  }
}
</script>
```

Los datos en DOM imperativo Vs Declarativo

En el DOM imperativo con JavaScript clásico nosotros, como programadores, tenemos que controlar paso a paso el DOM:

- Detectar cuándo cambian los datos (con eventos).
- Decidir qué elementos del HTML deben cambiar.
- Escribir todas las líneas necesarias para:
  - crear nodos (createElement)
  - modificar texto (textContent)
  - añadir clases (classList)
  - borrar elementos (remove, innerHTML = "")
  - insertar nodos (appendChild)
  - Mantener coherencia entre:
    - datos (arrays, objetos, variables)
    - interfaz (lo que ve el usuario)

Los datos en DOM imperativo Vs Declarativo

Problema de usar DOM imperativo

- **Si la interfaz crece, cada vez se hace más difícil saber qué tocar y cuando en el DOM y aparece el riesgo de:**
  - **Olvidar borrar un elemento**
  - **Duplicar filas**
  - **Que el estado visual no coincida con los datos**
  - **“Empantanar el código” con redundancias y bloques ineficientes**

En DOM imperativo estamos programando cómo cambiar la interfaz, no qué interfaz queremos.

Los datos en DOM imperativo Vs Declarativo

En el DOM declarativo con React nosotros no tenemos que crear ni borrar nodos:

- **Decimos: “Si el estado es este, la interfaz debe verse así.”**
- **React: compara el estado anterior con el nuevo**
- **Calcula qué ha cambiado**
- **Actualiza el DOM.**

En React nosotros gestionamos los datos, y React gestiona el DOM

React

- Es una librería de Javascript desarrollada por Facebook (Meta) y lanzada en 2013
- Utiliza enfoque declarativo y basado en componentes. Un componentes es una función que devuelve una interfaz.
- A partir del estado se actualiza la interfaz minimizando la manipulación directa del DOM → En React la interfaz es una función del estado
- Usa JSX una sintaxis especial para escribir interfaces usando javascript y con apariencia de HTML
- React sigue siendo una de las tecnologías frontend más usadas del mundo. (encuestas de StackOverflow).
- Lo utilizan: Meta, Netflix, Airbnb, Spotify, Uber, etc.
- El framework nextjs está basado en React

React

Estado

- Estado en React se refiere a los datos que cambian y afectan a la interfaz
- React gestiona el estado con *useState*.
- Son ejemplos de estado:
  - Los textos en los inputs
  - La lista de tareas
  - Si algo está visible o no

React

Render

- Render es el momento en que React dibuja (o re-dibuja) la interfaz en función del estado
- Cuando el estado cambia React vuelve a renderizar automáticamente.
- *Ya no es necesaria una función tipo "actualizarInterfaz()"*

React

props

- Props son datos que un componente recibe desde su componente padre
- Si el estado es lo que vive dentro de un componente, los props son los datos que entran desde fuera (el usuario los pasa).

React

Ejemplo: Formulario sencillo

App.tsx - nodebox - CodeSandl

codesandbox.io/p/sandbox/szsxcw?file=%2Fsrc%2FApp.tsx%3A14%2C20

CodeSandbox was updated! | Feedback | Sandbox | luisdelgado15sp / Drafts / Ejemplo

App.tsx | Preview

src > App.tsx > default

```
1  import { useState } from "react";
2
3  function App() {
4    const [nombre, setNombre] = useState("");
5
6    return (
7      <div>
8        <input value={nombre} onChange={(e) => setNombre(e.target.value)}
9        <p>Hola {nombre}</p>
10     </div>
11   );
12 }
13
14 export default App;
15
```

https://szsxcw.csb.app/

Hola

React

Ejemplo II: Lista

App.js

src > App.js > App > tareas.map() callback

```
1   import { useState } from "react";
2
3   export default function App() {
4     // Creamos un ESTADO con un array de las tareas, que son los datos principales de la app
5     const [tareas, setTareas] = useState([
6       "Poner negativos",
7       "Regalarle flores a Charo",
8     ]);
9
10    // Creamos otro ESTADO con el texto del input, lo que el usuario está escribiendo
11    const [texto, setTexto] = useState("");
12
```

https://6ywgvw.csb.app/

Lista de tareas (React)

Nueva tarea | Añadir tarea

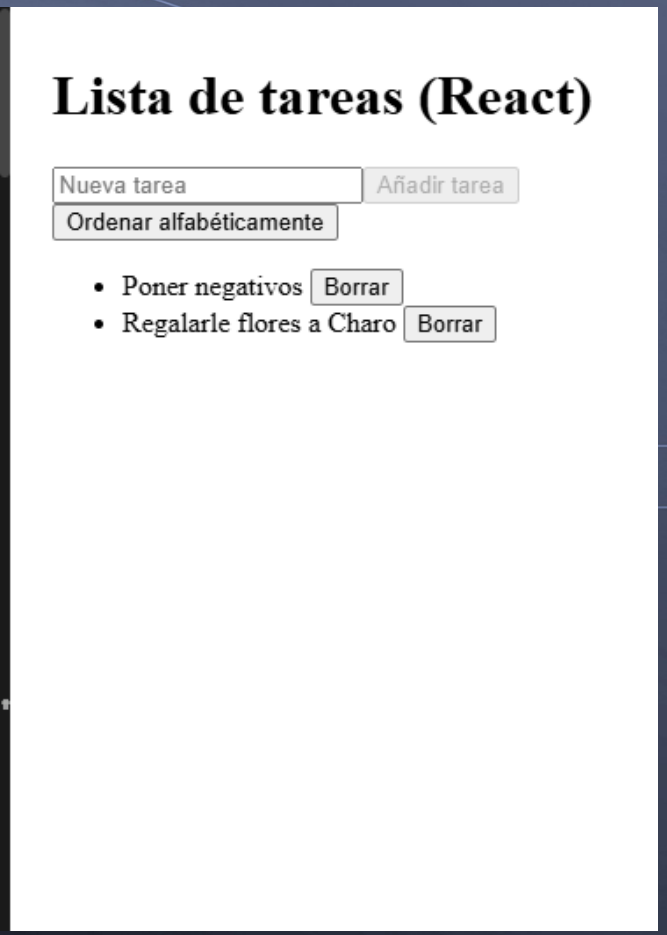
Ordenar alfabéticamente

- Poner negativos Borrar
- Regalarle flores a Charo Borrar

React

Ejemplo II: Lista

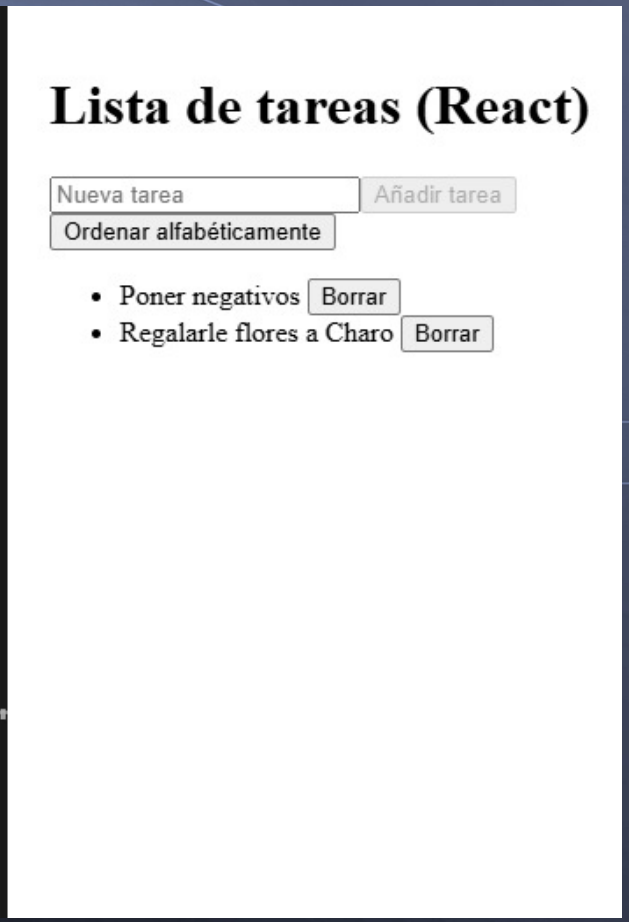
```
// Creamos una función "normalizar" para comparar strings donde:
// trim() elimina espacios al principio y al final
// toLowerCase() convierte a minúsculas
function normalizar(s) {
  return s.trim().toLowerCase();
}

// Creamos las siguientes variables, cuyo valor se
// recalcula en cada render.
// textoLimpio:  quita al texto del input los espacios sobrantes
// repetida: da true si la tarea ya existe (ignorando mayúsculas)
// incorrecta: valida que la tarea no está vacía ni (||) existe
// puedeAñadir: da true si la tarea NO es incorrecta
// puedeOrdenar: da true si hay 2 o más tareas (con length)
// (porque ordenar 0 o 1 tarea no tiene sentido)

const textoLimpio = texto.trim();

const repetida = tareas.some(
  (t) =>
    // El método some() comprueba si al menos un elemento del array
    // cumple con la condición implementada por la función proporcionada
    normalizar(t) === normalizar(textoLimpio)
);
const incorrecta = textoLimpio == "" || repetida;
const puedeAñadir = !incorrecta;
const puedeOrdenar = tareas.length > 1;
```



React

Ejemplo II: Lista → Create & Delete

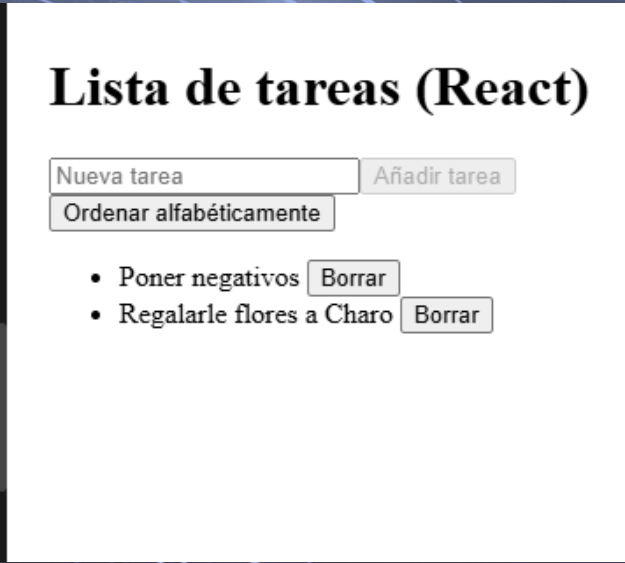
```
// CREATE: añadir tarea
// En React NO modificamos el array original (estado).
// Creamos un array nuevo con [...tareas, nuevaTarea]
// Los "..." en el array se llama operador de expansión (spread operator)
//sirve para crear una copia de los elementos del array (en este caso tareas)
function añadir() {
  // Aunque el botón "Añadir Tarea" esté deshabilitado, mantenemos una validación mínima
  // por seguridad por si un usuario cafre llamara a la función por consola.
  if (!puedeAñadir) return;

  // Actualizamos el estado con una copia + la nueva tarea
  setTareas([...tareas, textoLimpio]);

  // Limpiamos el input (estado texto) para que se puedan seguir añadiendo tareas
  setTexto("");
}

// DELETE: borrar tarea mediante un índice
// con el método filter creamos un array nuevo con todos los elementos
// excepto el que queremos eliminar

function borrarTarea(indice) {
  const copia = tareas.filter(function (t, i) {
    return i !== indice;
  });
  setTareas(copia);
}
```



React

Ejemplo II: Lista → Update

```
// ORDENAR: reordenar alfabéticamente
// Con sort() modificamos el array original,
// así que hacemos copia con [...tareas] antes de ordenar.
// localeCompare compara strings, devuelve un número de orden que indica
// si una cadena de carateres de referencia va antes, después o
// si es la misma que la cadena

function ordenar() {
  if (!puedeOrdenar) return;

  const copia = [...tareas];
  copia.sort(function (a, b) {
    return a.localeCompare(b);
  });
  setTareas(copia);
}
```



React

Ejemplo II: Lista → Render

```react
// RENDER (JSX):
// El siguiente return describe cómo debe verse la interfaz.
// Cuando cambia el estado (setTareas o setTexto), React vuelve a renderizar.

return (
  <div style={{ fontFamily: "inter", fontSize: 16, padding: 16 }}>
    <h1>Lista de tareas (React)</h1>

    {/* Ponemos la etiqueta del campo de INPUT con:value, que depende del estado "texto" y
        onChange que actualiza el estado "texto" con SetTexto*/}
    <input
      placeholder="Nueva tarea"
      value={texto}
      onChange={(e) => setTexto(e.target.value)}
    />

    {/* Botón Añadir tarea: disabled evita pulsar si está vacío o repetido      */}
    <button type="button" onClick={añadir} disabled={!puedeAñadir}>
      {" "}
      Añadir tarea{" "}
    </button>

    {/* Botón Ordenar:disabled si hay 0 o 1 tarea (no tiene sentido ordenar)      */}
    <button type="button" onClick={ordenar} disabled={!puedeOrdenar}>
      {" "}
      Ordenar alfabéticamente
    </button>
```

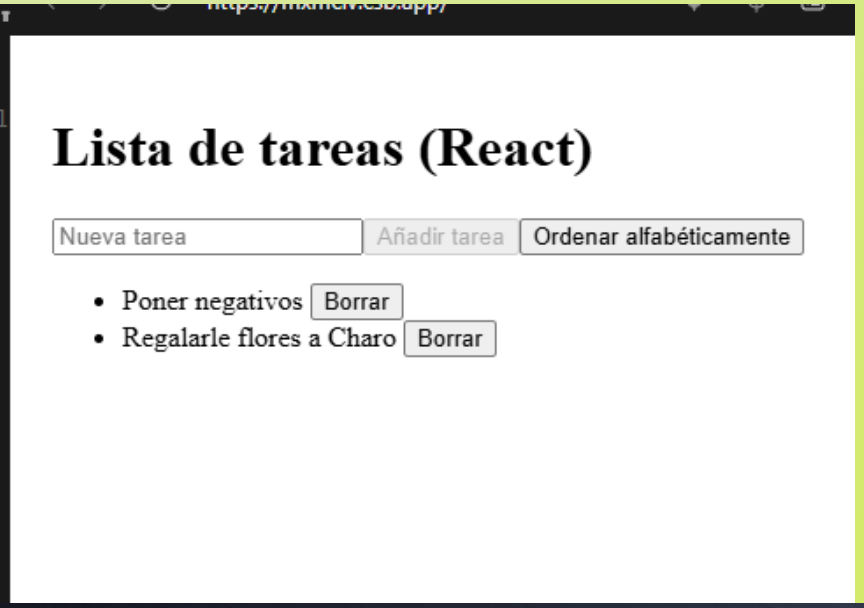
```react
export default function App() {

      {/* Feedback simple (no es obligatorio, pero ayuda a entender):si está repetida,
       mostramos una pista.      */}
      {repetida && <p>Esta tarea ya existe.</p>}

      {/* Lista:
          map convierte el array en elementos <li>
      */}
      <ul>
        {tareas.map(function (t, i) {
          return (
            <li key={i}>
              {t}{" "}
              <button type="button" onClick={() => borrarTarea(i)}>
                Borrar
              </button>
            </li>
          );
        })}
      </ul>
    </div>
  );
}
```

React

Cuando los componentes crecen

- Hasta ahora todo lo hemos hecho dentro de *App*. Aunque esto funciona, no es buena práctica sobre todo cuando una interfaz gana en complejidad y en funcionalidad.
- Vamos a dividir la interfaz en piezas
- Usaremos el ejemplo de la Lista con CRUD

```
import { useState } from "react";

export default function App() {
  // Creamos un ESTADO con un array de las tareas, que son los datos principales de l
  const [tareas, setTareas] = useState([
    "Poner negativos",
    "Regalarle flores a Charo",
  ]);

  // Creamos otro ESTADO con el texto del input, lo que el usuario está escribiendo
  const [texto, setTexto] = useState("");

  // Creamos una función "normalizar" para comparar strings donde:
  // trim() elimina espacios al principio y al final
  // toLowerCase() convierte a minúsculas
  function normalizar(s) {
    return s.trim().toLowerCase();
  }
```



React

componente hijo

- Vimos que un PROP es como un parámetro de una función
- Podemos crear un componente hijo mediante una función con props

```
function ItemTarea(props) {
  return (
    <li>
      {props.texto}
      <button onClick={props.onBorrar}>X</button>
    </li>
  );
}
```

- En este ejemplo props.texto es el dato introducido y props.onBorrar es una función que viene de fuera.

React

componente hijo

- Para usar el componente hijo dentro de la App sustituimos el bloque:

```
      <ul>
        {tareas.map(function (t, i) {
          return (
            <li key={i}>
              {t}{" "}
              <button type="button" onClick={() => borrarTarea(i)}>
                Borrar
              </button>
            </li>
          );
        })}
      </ul>
    </div>
  );
```

```
{tareas.map(function (t, i) {
  return (
    <ItemTarea
      key={i}
      texto={t}
      onBorrar={function () {
        borrarTarea(i);
      }}
    />
  );
})}
```

- key sigue siendo obligatorio.
- texto={t} envía el texto al componente.
- onBorrar envía una función.
- El componente hijo solo ejecuta

```
function ItemTarea(props) {
  return (
    <li>
      {props.texto}
      <button onClick={props.onBorrar}>X</button>
    </li>
  );
}
```

React

componente hijo

- **App tiene:**
  - ❑ el estado
  - ❑ las funciones

- **ItemTarea solamente:**
  - ❑ muestra cosas
  - ❑ lanza eventos

Para el ejemplo del formulario

App.jsx

src > App.jsx > ...

```
1  import { useState } from "react";
2
3  function Tarjeta(props) {
4    return <p>Hola {props.nombre}</p>;
5  }
6
7  export default function App() {
8    const [nombre, setNombre] = useState("");
9
10   return (
11     <div className="App">
12       <input value={nombre} onChange={(e) => setNombre(e.target.value)} />
13       <Tarjeta nombre={nombre} />
14     </div>
15   );
16 }
```

Preview

https://hfr2

Luis

Hola Luis

DDINT – 2 DAM

Luis.delgado15@educa.madrid.org